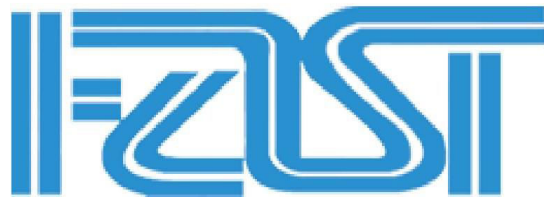


3/12/2023

CALCULUS AND ANALYTICAL GEOMETRY

Final Project Report



COURSE INSTRUCTOR: USMAN RASHID

Compiled by
Moiz Ansari (23i-0523) SEC-F
Minahil Kashif (23i-0554) SEC-F
Izza Urooj (23i-0669) SEC-F

Contents

Introduction	2
Objectives of the problem	2
By-Hand Solution	2
MATLAB Code	4
Explanation of Commands:	7
Step by Step Example of MATLAB Solution	7
Instruction Manual	9
Results	9
Flowchart of the Methodology	10
Conclusion	11
Contributions	11

Introduction

Our business customizes wood beams for optimal strength, utilizing customer-supplied raw wood. Using MATLAB and optimization, we identify precise dimensions, marrying traditional craftsmanship with modern efficiency.

Objectives of the problem

1. **Optimal Design:** Develop a program to find the strongest cross-sectional dimensions for wood beams from raw logs.
2. **Versatile Material Use:** Allow customers to use any wood, optimizing for strength, grain density, and resistance.
3. **Structural Efficiency:** Utilize MATLAB for an efficient interface that outputs the strongest beam dimensions from the input log diameter.

By-Hand Solution

$$d = 74$$

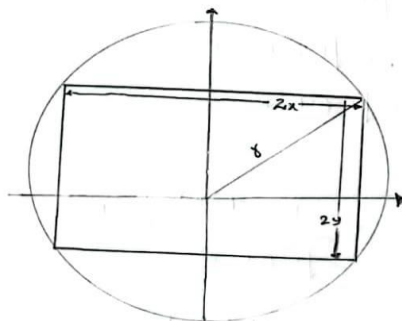
$$r = d/2 = 74/2$$

$$r = 37$$

By Using Pythagoras Theorem, we got

$$x^2 + y^2 = (37)^2$$

Now, the right upper corner of the rectangle will have the coordinates (x, y) . This means that the width of the rectangle will be $2x$ and height of the rectangle will be $2y$ as shown in below diagram.



The Area of rectangle then will be,

$$A = (2x)(2y) = 4xy$$

So, we've got the function we want to (maximize) the area. Since the coordinates of the upper right corner must be on the circle we know that x and y must satisfy the equation of circle.

The first thing to do then is to solve the constraint for one of the variables:

$$y^2 = 1369 - x^2$$

$$y = \sqrt{1369 - x^2}$$

Since that point we're looking at is in first quadrant we know that y must be positive and so, we can take the positive "+" part of this.

Plugging into the Area and computing the first derivative gives:

$$A = 2x \cdot 2\sqrt{1369 - x^2}$$

$$= 4x\sqrt{1369 - x^2}$$

diff w.r.t 'x' :-

$$\frac{d}{dx} A = 4\sqrt{1369 - x^2} + \frac{1}{2\sqrt{1369 - x^2}} \cdot (-2x) \cdot 4x$$

$$= 4\sqrt{1369 - x^2} - \frac{4x}{\sqrt{1369 - x^2}}$$

$$= \frac{4(\sqrt{1369 - x^2})(\sqrt{1369 - x^2}) - 4x}{\sqrt{1369 - x^2}}$$

$$= \frac{4(1369 - x^2) - 4x^2}{\sqrt{1369 - x^2}}$$

$$= \frac{5476 - 4x^2 - 4x^2}{\sqrt{1369 - x^2}} \Rightarrow \frac{5476 - 8x^2}{\sqrt{1369 - x^2}}$$

Now:- $\frac{dy}{dx}$ is not defined when:-

$$1369 - x^2 = 0$$

$$x = \pm 37$$

and is zero when:-

$$5476 - 8x^2 = 0$$

$$8x^2 = 5476$$

$$x^2 = 684.5$$

$$x = \pm 37\left(\frac{\sqrt{2}}{2}\right)$$

$$x = \pm 37/\sqrt{2} \text{ or } x = \pm 26.16$$

while Finding the Critical points, ranges in b/w
 $0 \leq x \leq 37$ which will lie in 1st quadrant. We
 got critical points as
 $x = +37 \quad \& \quad x = +37/\sqrt{2}$

Now, the area function is continuous and we have
 an interval of possible solution with finite Endpoints
 So,

$$A(37/\sqrt{2}) = 2737.9$$

$$A(37) = 0$$

So, we can see that we'll get the maximum
 area if $x = 37/\sqrt{2}$ and then the corresponding
 value of y will be

$$y = \sqrt{1369 - (37/\sqrt{2})^2} = 26.16$$

$$2y = 52.32$$

MATLAB Code

```
n=1; % Initializing variable to be used in loop
while n > 0 % Starting loop
    disp('WELCOME TO MMI Software House'); %Displaying names of software house
    disp("Programming Team: ");
    disp("1. Moiz Ansari (23i-0523)");
    disp("2. Minahil Kashif (23i-0554)");
    disp("3. Izza Urooj (23i-0669)");
    disp(" ");
    %Waiting for presskey:
    disp("Press any key to continue...");
    disp(" ");
    pause;
    %Taking input:
    diameter = input("Enter the Diameter of the log (in inches): ");
    disp(" ");
```

```

%Declaring variables:
syms x;
% Assuming x to be a real variable
assume (x, 'real');
%Radius of log:
r = diameter / 2;
%By using Pythagoras theorem:
 $x^2 + y^2 = r^2$ -----Eq1
%Area of rectangle:
 $A(x) = 4xy$ ;
%Using eq 1:
 $y(x) = \sqrt{\text{power}(r,2) - \text{power}(x,2)}$ ;
%Area:
 $A(x) = 4xy(x)$ ;
%Differntating A w.r.t x:
 $y1(x) = \text{diff}(A,x)$ ;
%Solving (derivative of A(x)) = 0:
%When  $A'(x) = 0$ ;
critical_point1 = solve(y1(x));
%Taking only poitive values:
result1 = abs(critical_point1);
%Checking range value for critical point 1:
critical_point1 = abs(critical_point1);
if critical_point1<=r
    result1 = critical_point1;
else
    result1 = 0;
end
%Printing critical point 1:
disp("Critical point 1: ");
disp(result1);
%When  $A'(x)$  is not defined:
critical_point2 = solve(y);
%Taking only poitive values:
critical_point2 = abs(critical_point2);

```

```

%Checking range value for critical point 2:
if critical_point2<=r
    result2 = critical_point2;
else
    result2 = 0;
end
%Printing critical point 2:
disp("Critical point 2: ");
disp(result2);
%Checking which critical point gives larger area:
%...
n1 = A(result1);
n2 = A(result2);
if n1>n2
    X = result1;
else
    X = result2;
end
%...
%Calculating y dimensions:
Y = sqrt(power(r,2)-power(X,2));
dim_y = 2*Y;
dim_x = 2*X;
%Printing Dimensions:
disp("Dimension x = ");
disp(dim_x);
disp("Dimension y = ");
disp(dim_y);
%Asks user if run program again or not:
reply = input ('Do you wish to run another query? Y/N [Y]:', 's');
if strcmpi(reply,'Y') %Ignores case sensitive
    n=1;
else
    n=0;
end
end

```

Explanation of Commands:

1. while : loop used to run the program again and again
2. disp : to display the text and results
3. pause: for execution until a key is pressed.
4. input : to take input from user
5. if else : to check if a condition is true or not
6. syms: to declare symbolic variables
7. assume : to assume that the variable is a real number
8. diff: to find the derivative of the equation
9. solve: to solve the equation at a particular value
10. strcmpi : to compare two strings, ignoring case sensitivity.

Step by Step Example of MATLAB Solution

1. We initialize our variable n and initiate a while loop which will iterate throughout the code. Displaying the names of the group members and asking the user to press a key that makes the code start running again.

```
n=1; % Initializing variable to be used in loop
while n > 0 % Starting loop
    disp('WELCOME TO MMI Software House'); %Displaying names of software house
    disp("Programming Team: ");
    disp("1. Moiz Ansari (23i-0523)");
    disp("2. Minahil Kashif (23i-0554)");
    disp("3. Izzah Urooj (23i-0669)");
    disp(" ");

    %Waiting for presskey:
    disp("Press any key to continue...");
    disp(" ");
    pause;
```

➤ OUTPUT

```
>> project
WELCOME TO MMI Software House
Programming Team:
1. Moiz Ansari (23i-0523)
2. Minahil Kashif (23i-0554)
3. Izzah Urooj (23i-0669)

Press any key to continue...
```

2. Taking Input of the diameter of the log and calculating the radius.

```
%Taking input:
diameter = input("Enter the Diameter of the log (in inches): ");
disp(" ");

%Declaring variables:
syms x;

% Assuming x to be a real variable
assume(x, 'real');

%Radius of log:
r = diameter / 2;
```

➤ OUTPUT

```
Enter the Diameter of the log (in inches):
74
```

3. - Declaring Pythagoras theorem as eq1 and Area of Rectangle as A(x)
-Taking the derivative of A(x) w.r.t x and equating it to 0 to find the critical point.

-Checking if it lies within the possible range if it does it is one side of the Rectangle and we print it, otherwise it is not.

```
%By using pythagoras theorem:
%x^2 + y^2 = r^2-----Eq1

%Areaofrectangle:
%A(x) = 4*x*y;

%Using eq 1:
y(x) = sqrt(power(r,2)-power(x,2));
%Area:
A(x) = 4*x*y(x);

%Differentiating A w.r.t x:
y1(x) = diff(A,x);

%Solving (derivative of A(x)) = 0:
%When A'(x) = 0;
critical_point1 = solve(y1(x));

%Taking only poitive values:
result1 = abs(critical_point1);

%Taking only poitive values:
result1 = abs(critical_point1);

%Checking range value for critical point 1:
critical_point1 = abs(critical_point1);
if critical_point1<=r
    result1 = critical_point1;
else
    result1 = 0;
end

%Printing critical point 1:
disp("Critical point 1: ");
disp(result1);
```

4. Solve y(x) and find the 2nd critical point

```
%Taking only poitive values:
critical_point2 = abs(critical_point2);

%Checking range value for critical point 2:
if critical_point2<=r
    result2 = critical_point2;
else
    result2 = 0;
end

%Printing critical point 2:
disp("Critical point 2: ");
disp(result2);
```

```
Critical point 1:
(37*2^(1/2))/2
(37*2^(1/2))/2
```

```
Critical point 2:
37
37
```

➤ OUTPUT

5. Checking which point gives the larger area and using that to calculate the dimensions.
6. Prompting the user to check if they want to run the program again.

```

%Checking which critical point gives larger area:
%...
n1 = A(result1);
n2 = A(result2);
if n1>n2
    X = result1;
else
    X = result2;
end

%Caclulating y dimensions:
Y = sqrt(power(r,2)-power(X,2));
dim_y = 2*Y;
dim_x = 2*X;

%Printing Dimensions:
disp("Dimension x = ");
disp(dim_x);
disp("Dimension y = ");
disp(dim_y);

%Asks user if run program again or not:
reply = input('Do you wish to run another query? Y/N [Y]:', 's');
if strcmpi(reply,'Y') %Ignores case sensitive
    n=1;
else
    n=0;
end
end

```

➤ OUTPUT

```

Dimension x =
37*2^(1/2)
37*2^(1/2)

Dimension y =
37*2^(1/2)
37*2^(1/2)

Do you wish to run another query? Y/N [Y]:
n

```

Instruction Manual

- First press any key to start the program.
- Then enter the diameter and press enter.
- The result will then appear on the command window.
- Now if you want to run the program again and don't want to quit then press 'y'.
- Otherwise press 'n' to exit the program.
- If you choose to start the program again, then the same procedure is repeated.

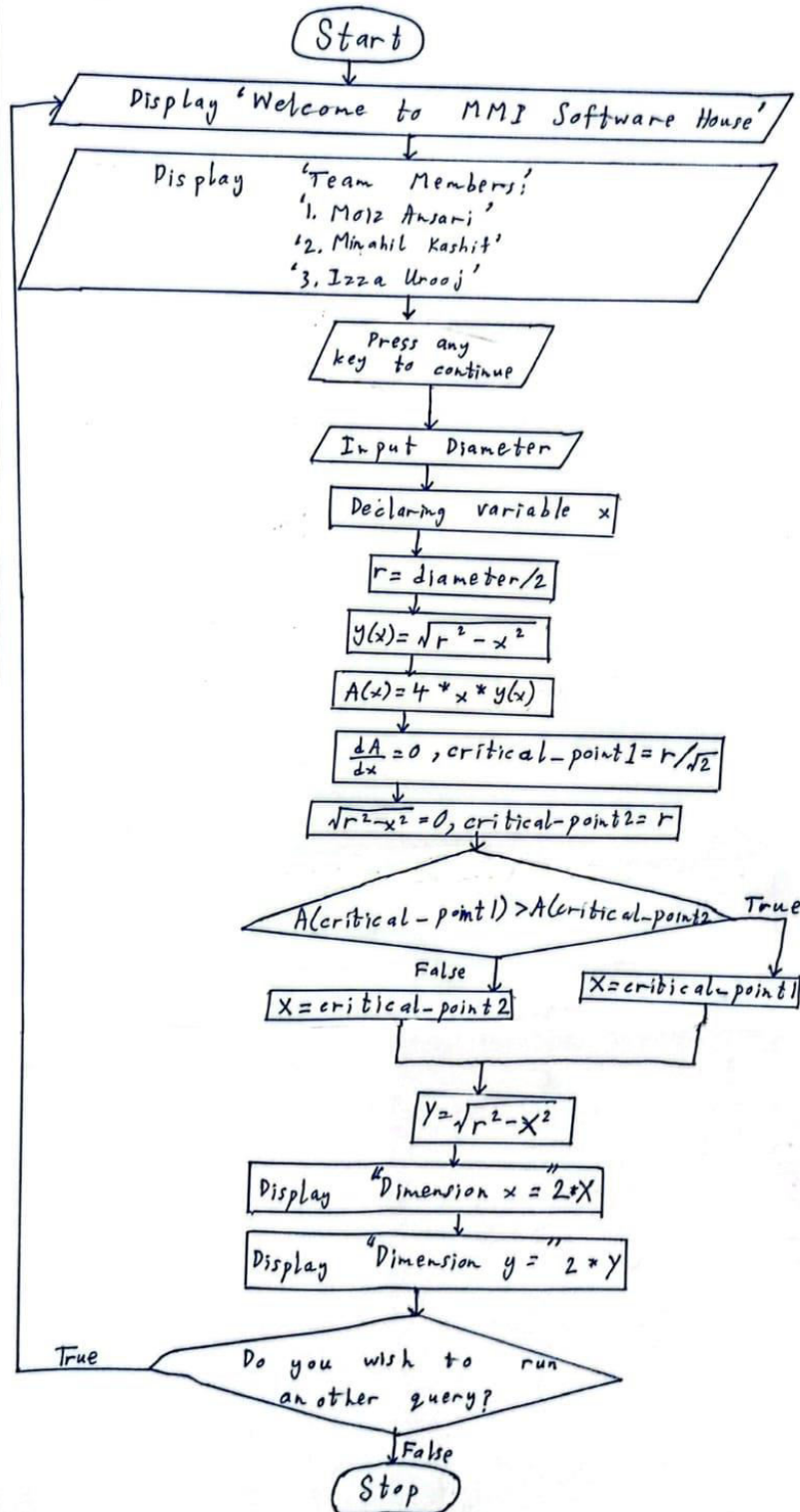
Results

The program result is the dimensions of cross-section of the strongest beam possible to be cut from the log.

Our results were the same as the by hand solution.

Flowchart of the Methodology

Flowchart:



Conclusion

- **Summary of the problem**

The problem was about optimization. We had to use optimization techniques to find the best possible way to make a beam out of a wooden log.

- **Results**

We used 2 equations, derivated them, and evaluated them to find the cross-section of the strongest wooden beams.

- **Difficulties faced**

We faced diffuculties in finding the commands to make the equations and to find the critical points. We researched online to find the required commands.

Contributions

All 3 of us formulated the working of the problem together.

- Izzah Urooj wrote the final step-by-step solution of the problem.
- Moiz Ansari wrote the codes for the loops, the equations, and the derivations in the MATLAB program
- Minahil Kashif demonstrated the MATLAB solution step-by-step using an example.